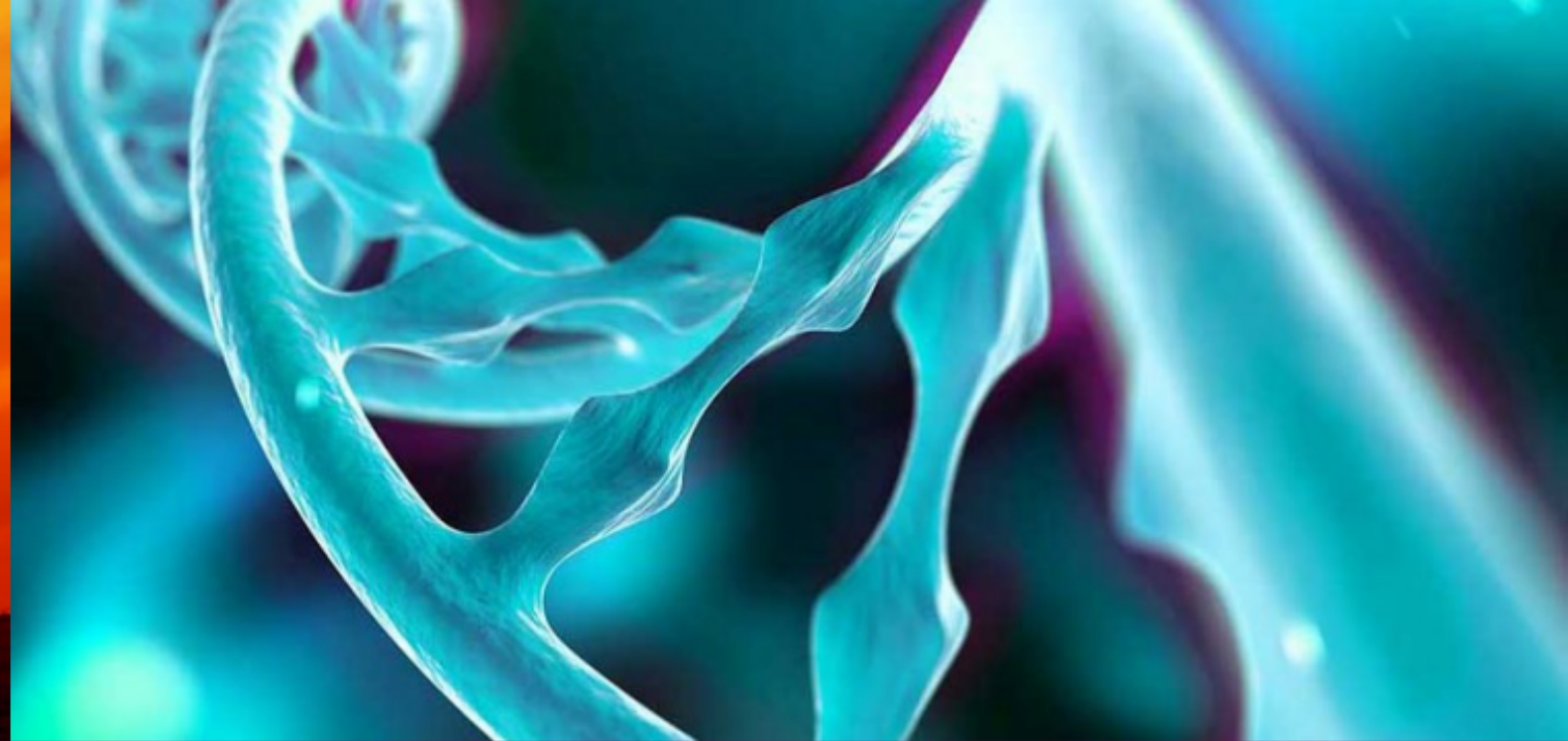
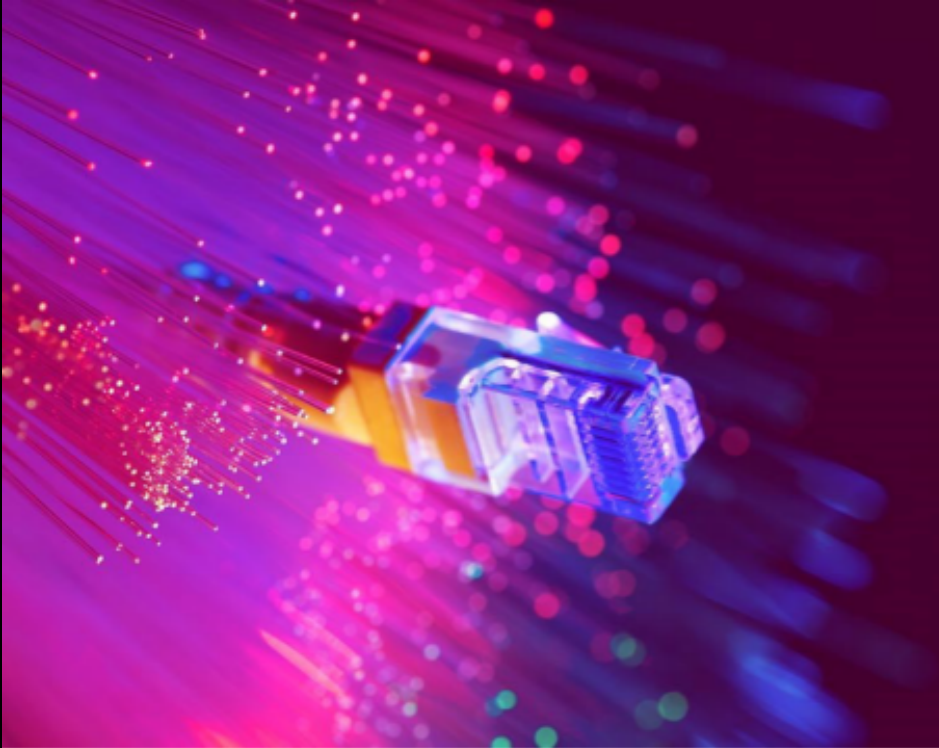




Parallel computing with threads

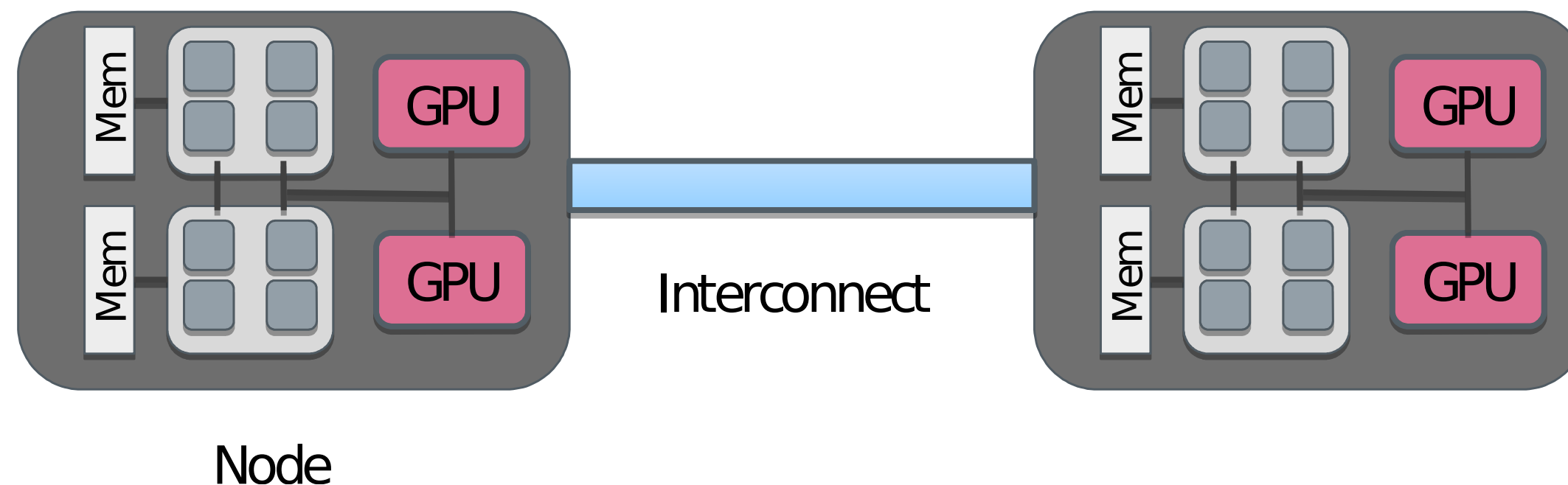
Student Cluster Competition 2024



CSC – Finnish expertise in ICT for research, education and public administration

Anatomy of supercomputer

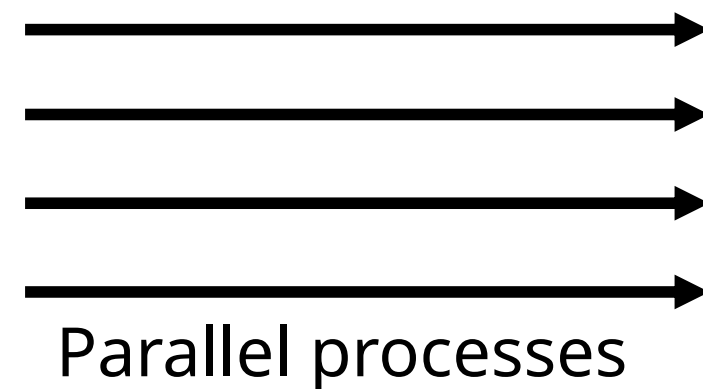
- Supercomputers consist of nodes connected with high-speed network
 - Latency $\sim 1 \mu\text{s}$, bandwidth $\sim 200 \text{ Gb / s}$
- A node can contain several multicore CPUs
- Additionally, nodes can contain one or more accelerators
- Memory within the node is directly usable by all CPU cores



Parallel programming models

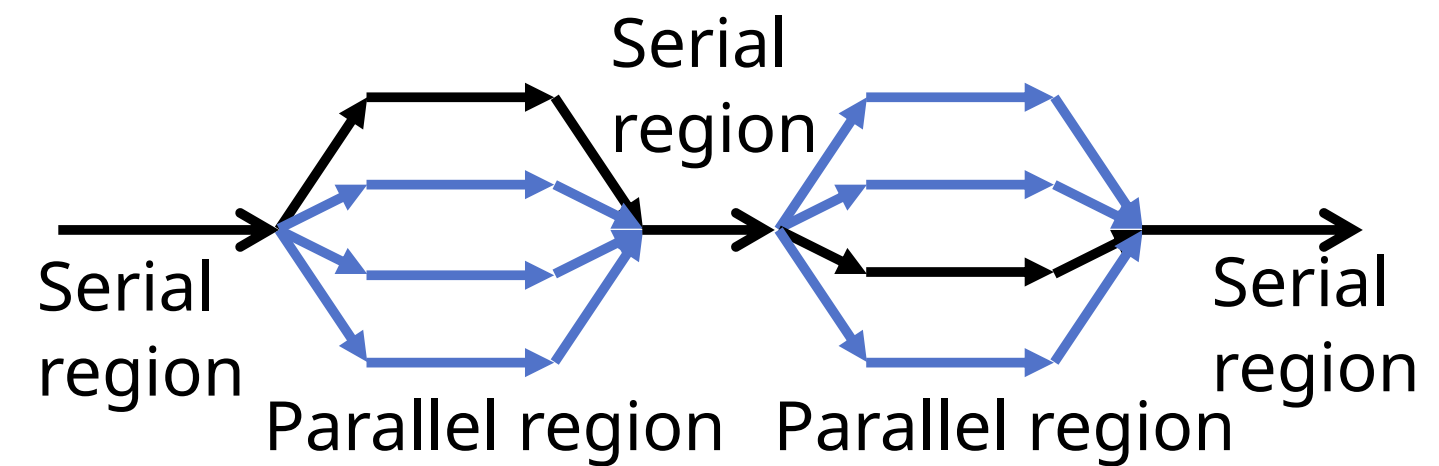
- Parallel execution is based on threads or processes (or both) which run at the same time on different CPU cores
- Processes
 - Interaction is based on exchanging messages between processes
 - MPI (Message passing interface)
- Threads
 - Interaction is based on shared memory, i.e. each thread can access directly other threads data
 - OpenMP, pthreads
 - (CUDA, HIP, OpenACC)

Parallel programming models - CPUs



MPI: Processes

- Independent execution units
- MPI launches N processes at application startup
- Works over multiple nodes

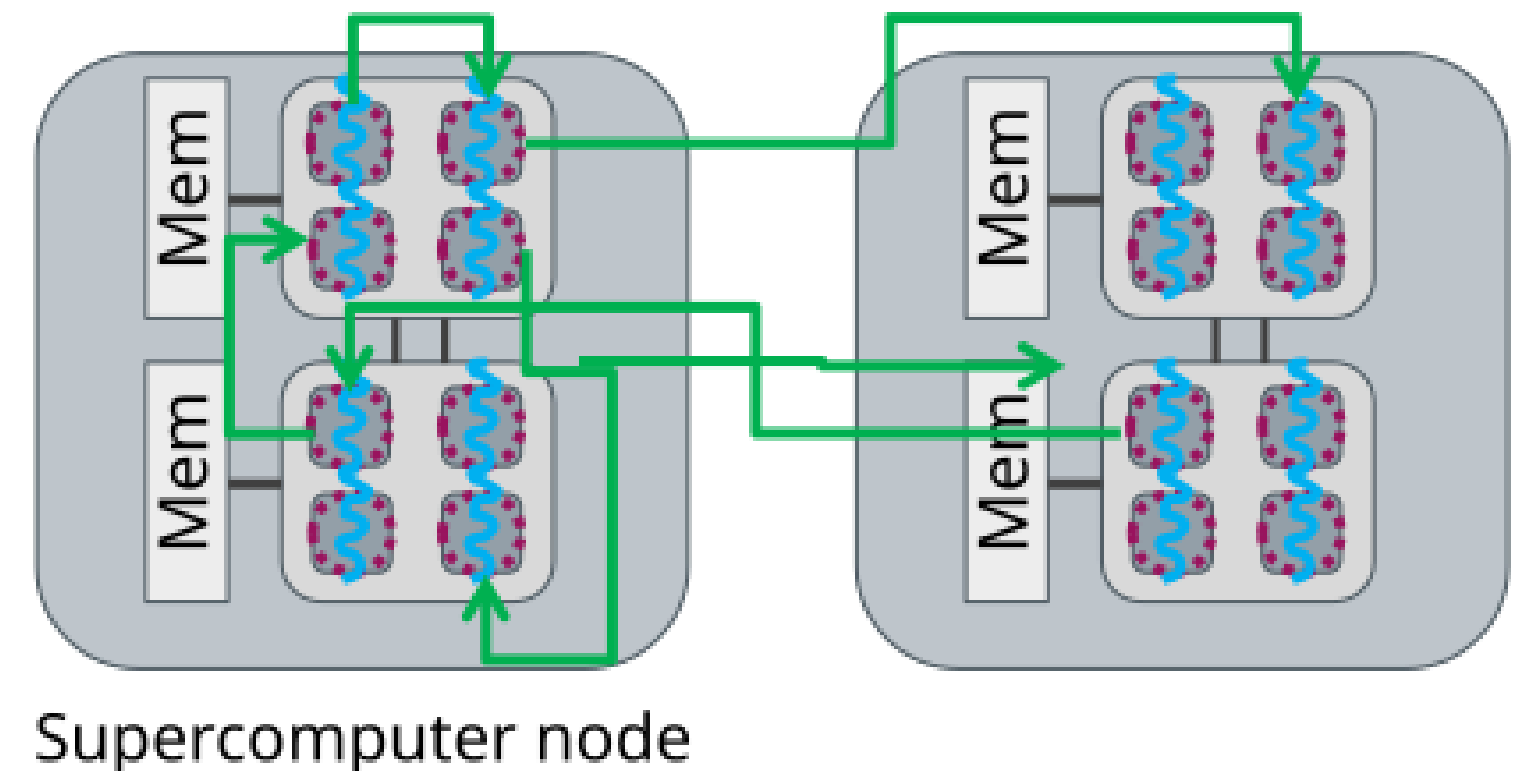


OpenMP: Threads

- Threads share memory space
- Threads are created and destroyed (parallel regions)
- Limited to a single node

Hybrid programming: Launch threads (OpenMP) *within* processes (MPI)

- Shared memory programming inside a node, message passing between nodes
 - Matches well modern supercomputer hardware
 - Optimum MPI task per node ratio depends on the application and should always be experimented.



- MPI task
- ~ OpenMP thread
- MPI message

Running parallel programs

- Most parallel programs are written to work with arbitrary number of processes / threads / GPUs
- Number of processes / threads / GPUs is specified when launching the program
 - Number of threads within GPU is typically given inside the program
- Number of processes: specific launch command (srun)
- Number of threads: environment variables (OMP_NUM_THREADS)

Thread and process affinity

Thread and process affinity

- Normally, operating system can run threads and processes in any logical core
- Operating system may even move running task from one core to another
 - Can be beneficial for load balancing
 - For HPC workloads often detrimental as private caches get invalidated and NUMA locality is lost
- User can control where tasks are run via affinity masks
 - Task can be *pinned* to a specific logical core or set of logical cores

Controlling affinity

- Affinity for a *process* can be set with a `numactl` command
 - Limit the process to logical cores 0,3,7:
`numactl --physcpubind=0,3,7 ./my_exe`
 - Threads “inherit” the affinity of their parent process
- Affinity of a thread can be set with OpenMP environment variables
 - `OMP_PLACES=[threads,cores,sockets]`
 - `OMP_PROC_BIND=[true, close, spread, master]`
- OpenMP runtime prints the affinity with `OMP_DISPLAY_AFFINITY=true`

Controlling affinity

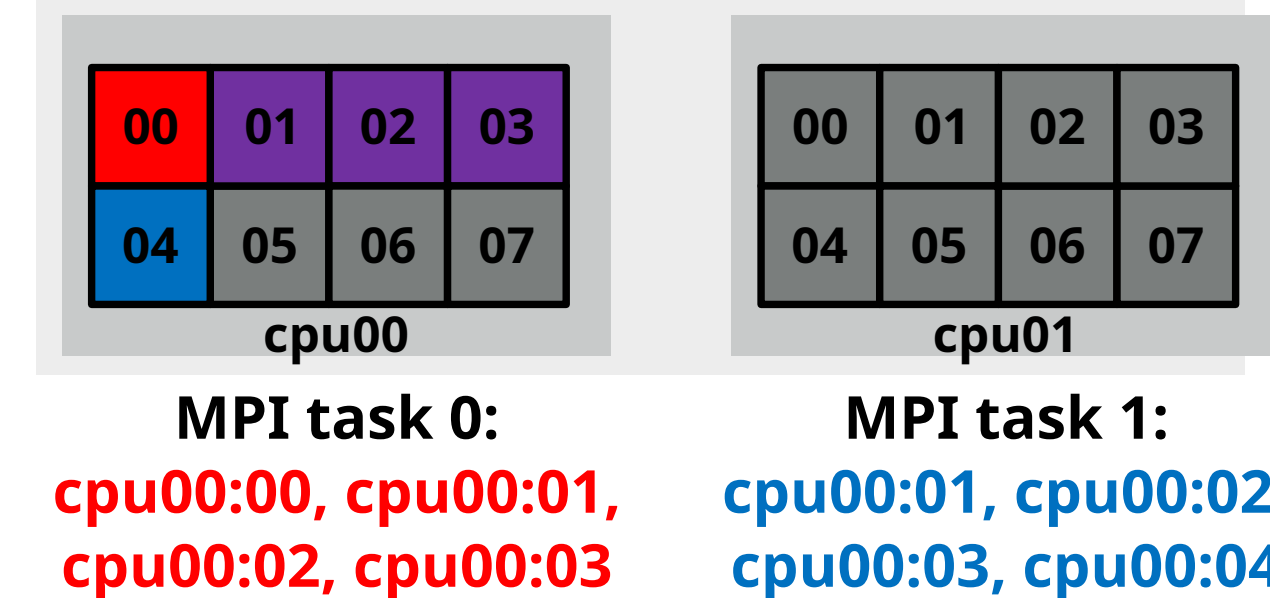
```
export OMP_AFFINITY_FORMAT="Thread %0.3n affinity %A"  
export OMP_DISPLAY_AFFINITY=true  
./test  
Thread 000 affinity 0-7  
Thread 001 affinity 0-7  
Thread 002 affinity 0-7  
Thread 003 affinity 0-7
```

```
OMP_PLACES=cores ./test  
Thread 000 affinity 0,4  
Thread 001 affinity 1,5  
Thread 002 affinity 2,6  
Thread 003 affinity 3,7
```

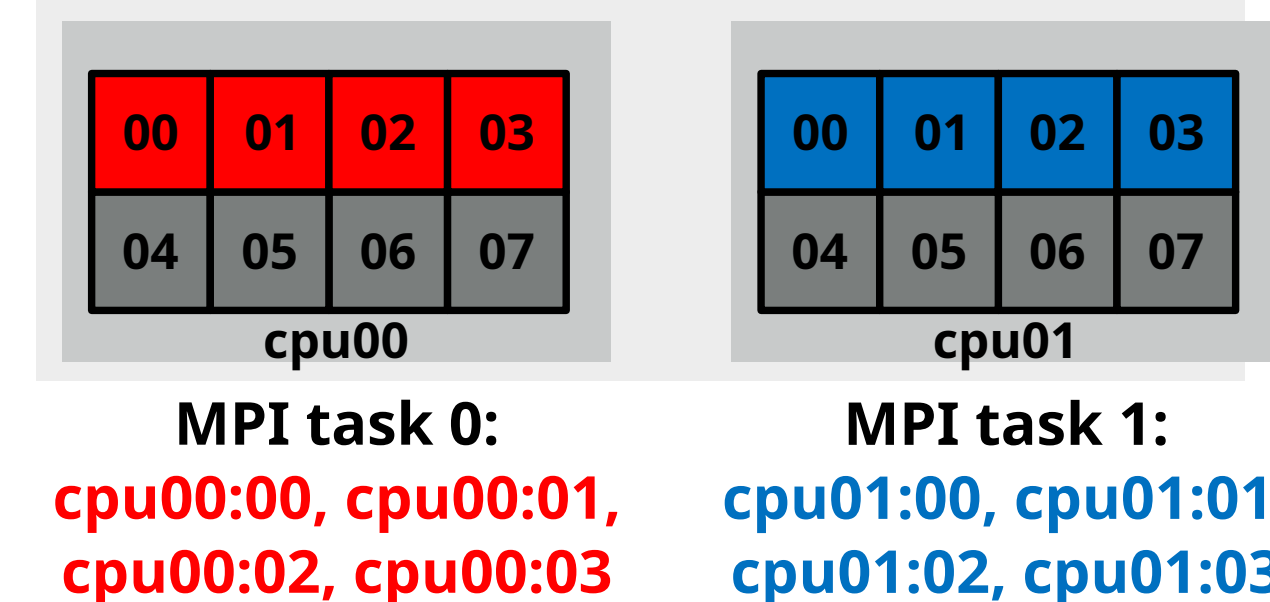
MPI+OpenMP thread affinity

- MPI library must be aware of the underlying OpenMP for correct allocation of resources
 - Oversubscription of CPU cores may cause significant performance penalty
- Additional complexity from batch job schedulers
- Heavily dependent on the platform used!

Example (**incorrect**): oversubscription of resources



Example (**correct**): better use of resources



Slurm configuration at CSC

- Within a node, `--tasks-per-node` MPI tasks are spread `--cpus-per-task` apart
- Threads within a MPI tasks have the affinity mask for the corresponding `--cpus-per-task` cores

```
export OMP_AFFINITY_FORMAT="Process %P thread %0.3n affinity %A"  
export OMP_DISPLAY_AFFINITY=true  
srun ... --tasks-per-node=2 --cpus-per-task=4 ./test  
Process 250545 thread 000 affinity 0-3  
...  
Process 250546 thread 000 affinity 4-7  
...
```

- Slurm configurations in other HPC centers can be very different
 - Always experiment before production calculations!

Summary

- Performance of HPC applications is often improved when processes and threads are pinned to CPU cores
- MPI and batch system configurations may affect the affinity
 - very system dependent, try to always investigate